



Desafio Final

Solicitação do Cliente

Olá, meu nome é Tyler e sou representante da **Grid Motors**, uma concessionária especializada em locação de veículos.

Estamos procurando um parceiro para desenvolver o backend do nosso sistema.

Devido à urgência, teremos apenas uma única entrega. Para isso, pensamos em desenvolver funcionalidades para cadastro de:

- Carros
- Reservas
- Usuários
- Autenticação

Características do desafio

O desafio consiste em desenvolver uma **API REST completa**.

Requisitos Obrigatórios

- Código legível
- Repositório privado
- Commits pequenos
- Explicar como executar o projeto localmente
- Qualquer Banco de dados
- Docker compose com seu banco de dados para facilitar rodar local
- ESLint e Prettier
- Swagger disponível em:
- http://localhost:3000/api/v1/api-docs
- Cobertura de testes mínima de **60%** (sugestão: utilizar Jest)
- Todas as rotas de **Carros** devem exigir **Bearer Token**

Critérios de Avaliação

- Qualidade do código
- Organização do projeto
- Qualidade da API REST
- Lógica da solução implementada
- Arquitetura do projeto
- Qualidade da camada de persistência
- Utilização do Git (quantidade, descrição dos commits e Git Flow)

> Observação: ao final deste documento há alguns links úteis para auxiliar no desenvolvimento.

Git & GitHub

- Crie um repositório privado no GitHub.
- Siga um padrão de commits consistente (Conventional Commits:

`feat:`, `fix:`, `refactor:`)

- **Commits curtos, descritivos e em inglês**
- **Adicionar instrutores e SM como membros**
  a. Anthoni: anthoni.martinelli@compasso.com.br
  b. Ariel: ariel.souza@compasso.com.br
  c. Jailson: jailson.souza@compasso.com.br
  d. Antônio: antonio.carvalho@compasso.com.br
  e. Gilson: gilson.junior@aircompany.ai
  f. Crisciano: crisciano.botelho@aircompany.ai

Endpoints

🚙 Carros

1. Cadastrar um carro

Request Body

JavaScript

```
{
  "model": "GM S10 2.8",
  "color": "White",
  "year": "2021",
  "value_per_day": 50,
  "accessories": [
    {
      "description": "air conditioner"
    },
    {
      "description": "4x4 traction"
    },
    {
      "description": "4 ports"
    }
  ],
  "number_of_passengers": 5
```

```
}
```

Response Body

JavaScript

```
{
  "model": "GM S10 2.8",
  "color": "White",
  "year": "2021",
  "value_per_day": 50,
  "accessories": [
    {
      "description": "air conditioner"
    },
    {
      "description": "4x4 traction"
    },
    {
      "description": "4 ports"
    }
  ],
  "number_of_passengers": 5
}
```

Regras

- Todos os campos são obrigatórios
- Deve existir pelo menos **um acessório**
- O ano de fabricação deve estar entre **1950 e 2023**
- Não pode haver acessórios repetidos
- Em caso de sucesso retornar **201** com o carro cadastrado
- Caso existam campos inválidos retornar **400** informando o erro

---

2. Listar todos os carros

Response Body

JavaScript

```
{
```

```
  "car": [
    {
      "id": "asdasda149e5bda4f23e25a12d396a07fd3098",
      "model": "GM S10 2.8",
      "color": "White",
      "year": "2021",
      "value_per_day": 50,
      "accessories": [
        {
          "description": "air conditioner"
        },
        {
          "description": "4x4 traction"
        },
        {
          "description": "4 ports"
        }
      ],
      "number_of_passengers": 5
    }
  ],
  "total": 1000,
  "limit": 100,
  "offset": 1,
  "offsets": 10
}
```

Regras

- Deve ser possível pesquisar utilizando **query params**
- Exemplo:
  - pesquisar todos os carros de determinado modelo
- A pesquisa deve funcionar para qualquer campo do carro
- Deve possuir paginação
- Em caso de sucesso retornar **200** com a lista de carros

---

3. Remover um carro

Regras

- Em caso de sucesso retornar **204** sem conteúdo
- Se o ID possuir formato inválido retornar **400**

- Caso o ID não exista retornar **404**

---

4. Atualizar um carro

Request Body

JavaScript

```
{
  "model": "GM S10 2.8",
  "color": "White",
  "year": "2021",
  "value_per_day": 50,
  "accessories": [
    {
      "description": "air conditioner"
    },
    {
      "description": "4x4 traction"
    },
    {
      "description": "4 ports"
    }
  ],
  "number_of_passengers": 5
}
```

Response Body

JavaScript

```
{
  "model": "GM S10 2.8",
  "color": "White",
  "year": "2021",
  "value_per_day": 50,
  "accessories": [
    {
      "description": "air conditioner"
    },
    {
      "description": "4x4 traction"
    },
    {
```

```
      "description": "4 ports"
    }
  ],
  "number_of_passengers": 5
}
```

Regras

- Qualquer campo pode ser alterado, exceto o **ID**
- Em caso de sucesso retornar **200** com o carro atualizado
- Se o ID possuir formato inválido retornar **400**
- Caso o ID não exista retornar **404**
- Aplicam-se as mesmas validações do cadastro

---

5. Buscar carro pelo ID

Response Body

JavaScript

```
{
  "model": "GM S10 2.8",
  "color": "White",
  "year": "2021",
  "value_per_day": 50,
  "accessories": [
    {
      "description": "air conditioner"
    },
    {
      "description": "4x4 traction"
    },
    {
      "description": "4 ports"
    }
  ],
  "number_of_passengers": 5
}
```

Regras

- Em caso de sucesso retornar **200**

- Se o ID possuir formato inválido retornar **400**
- Caso o ID não exista retornar **404**

👤 Usuários

Assim como os carros, o usuário deve possuir operações de:

- Cadastro
- Atualização
- Exclusão
- Busca por ID
- Listagem

---

6. Cadastrar usuário

Request Body

JavaScript

```
{
  "name": "Joãozinho Ciclano",
  "cpf": "131.147.860-49",
  "birth": "03/03/2000",
  "email": "joazinho@email.com",
  "password": "123456",
  "cep": "01001000",
  "qualified": "sim"
}
```

No cadastro será enviado apenas o **CEP**.

O endereço completo deverá ser obtido através da API **ViaCEP**.

Endpoint: https://viacep.com.br

Dicas

1. Utilize Template String ou `.replace()` para montar a URL.
2. Utilize a biblioteca **Axios**.
3. Considere os dados retornados até o campo **uf**.
4. O payload está demonstrado no PDF.

Regras

- Todos os campos são obrigatórios
- O usuário deve possuir pelo menos **18 anos**
- CPF válido
- CPF único
- Email único
- Senha com no mínimo **6 caracteres**
- Email válido
- O campo **qualified** aceita apenas:
  - yes
  - no
- Em caso de sucesso retornar **201**
- Em caso de erro retornar **400**

---

7. Autenticação

Request Body

JavaScript

```
{
  "email": "joazinho@email.com",
  "password": "123456",
}
```

Regras

- Em caso de sucesso retornar **200** com o token
- Caso as credenciais sejam inválidas retornar **400**

---

8. Atualizar acessórios do carro

Esta rota deve permitir:

- adicionar um acessório
- remover um acessório

Request Body

JavaScript

```
{
  "descricption": "Air Conditioner",
}
```

Regras

- O campo **description** é obrigatório
- Em caso de sucesso retornar **200**
- Se o ID possuir formato inválido retornar **400**
- Caso o ID não exista retornar **404**

Dica

Se o acessório já existir:

- remover

Caso contrário:

- adicionar

Reservas

9. Criar uma reserva

Request Body

JavaScript

```
{
  "start_date": "20/04/2023",
  "end_date": "30/04/2023",
  "id_car": "asdasda149e5bda4f23e25a12d393asd30910",
}
```

Observações

- `id_user`: usuário responsável pela reserva
- `id_car`: carro reservado

Regras

- Todos os campos são obrigatórios
- Calcular `final_value` utilizando `value_per_day`
- Apenas usuários habilitados podem reservar
- Não pode existir mais de uma reserva do mesmo carro para o mesmo dia
- Um usuário não pode possuir reservas sobrepostas

Exemplo

Se um usuário reservou um carro de:

20/03/2023

até

30/03/2023

ele não poderá reservar qualquer outro carro dentro desse período.

Em caso de sucesso retornar **201**.

Caso haja erro retornar **400**.

---

10. Listar reservas

Response Body

JavaScript

```
{
  "reserves": [
    {
      "id_reserve":
"asdasda149e5bda4f23e25a12d396a07fd3098",
      "id_user":
"asdasda149e5bda4f23e25a12d396a07fd3098",
      "start_date": "20/04/2023",
      "end_date": "30/04/2023",
      "id_car":
"asdasda149e5bda4f23e25a12d393asd30910",
      "final_value": "500,00"
    },
    {
      "id_reserve":
"asdasda149e5bda4f23e25a12d396a07fd3098",
      "id_user":
"asdasda149e5bda4f23e25a12d396a07fd3098",
      "start_date": "20/04/2023",
      "end_date": "30/04/2023",
      "id_car":
"asdasda149e5bda4f23e25a12d393asd30910",
      "final_value": "500,00"
    }
  ],
  "total": 1000,
  "limit": 100,
  "offset": 1,
  "offsets": 10
}
```

Observações

- `id_user`: usuário responsável
- `id_car`: carro reservado

Regras

Deve ser possível pesquisar por qualquer campo utilizando query params.

Exemplos

- valor final
- nome

- id_user
- id_car
- start_date
- end_date
- final_value

Também deve possuir paginação.

---

11. Buscar reserva pelo ID

Response Body

JavaScript

```
{
  "id_reserve": "asdasda149e5bda4f23e25a12d396a07fd3098",
  "id_user": "asdasda149e5bda4f23e25a12d396a07fd3098",
  "start_date": "20/04/2023",
  "end_date": "30/04/2023",
  "id_car": "asdasda149e5bda4f23e25a12d393asd30910",
  "final_value": "500,00"
}
```

---

12. Atualizar reserva

Regras

- Todos os campos podem ser alterados
- Se o ID possuir formato inválido retornar **400**
- Aplicam-se as mesmas regras do cadastro
- Não é permitido atualizar uma reserva caso o carro já possua outra reserva para a data seguinte

---

13. Remover reserva

Regras

- Em caso de sucesso retornar **204**
- Se o ID possuir formato inválido retornar **400**
- Caso o ID não exista retornar **404**

Links úteis

- http://karma-runner.github.io/4.0/dev/git-commit-msg.html
- https://dev.to/dianaops/como-escrever-um-readme-md-sensacional-no-github-4509
- https://desenvolvimentoparaweb.com/javascript/map-filter-reduce-javascript/
- https://www.youtube.com/watch?v=KKTX1l3sZGk
- https://www.youtube.com/watch?v=rCeGfFk-uCk
- https://expressjs.com/pt-br/guide/writing-middleware.html
- https://www.npmjs.com/package/swagger-ui-express
- https://github.com/rocketseat-content/youtube-nodejs-tdd-jest